

BigQuery Advanced Handbook

External Tables, Pricing, IAM, BQML, Time Travel, Vector Search, Lineage & Security

This handbook explains ten advanced BigQuery capabilities in depth — external tables and BigLake/Lakehouse for Iceberg, the pricing model and slot reservations, IAM and data access control, BigQuery ML, time travel and change history, Vector Search for RAG, remote functions, data lineage, row-level security, and encryption — each with the business problem it solves, the concepts underneath, working SQL/CLI, and exactly how to verify the result in the console.

Problem statement: "ShopIndex," an e-commerce company, has outgrown basic load-query-join usage of BigQuery. Some data lives outside BigQuery entirely; the bill is unpredictable; an audit needs proof of who can see what and where a number came from; and the data science team wants to build models and a semantic search feature without ever exporting data out of the warehouse. This handbook builds up exactly the techniques needed to answer each of those needs, using one running dataset (customers, products, orders) throughout.

1. External Tables, BigLake & Lakehouse for Apache Iceberg

The problem: not all of ShopIndex's data should be copied into BigQuery-managed storage — some of it is maintained by other teams' pipelines (Spark jobs writing Parquet, partners dropping files into a shared bucket). Copying it in means paying for duplicate storage and it goes stale the moment the source updates.

1.1 Plain External Tables — Query GCS Data In Place

```
CREATE OR REPLACE EXTERNAL TABLE `dataset.orders_external`  
OPTIONS (  
  format = 'NEWLINE_DELIMITED_JSON',  
  uris = ['gs://bucket/orders.json']  
);  
  
SELECT order_id, order_date, status FROM `dataset.orders_external` LIMIT 10;
```

No data is copied. The table is a reference only — bytes stay in GCS. Without a WITH CONNECTION clause, this is a plain external table: query-time-only IAM enforcement, no fine-grained access control.

Check it in the console: Table's Details tab shows Table type: EXTERNAL and a Source URI(s) field pointing at GCS. Storage stats show ~0 bytes, confirming nothing was copied.

1.2 Managed Apache Iceberg Tables — BigLake / "Lakehouse for Apache Iceberg"

The difference from 1.1: a managed Iceberg table is a full BigQuery-managed table that happens to store data as Parquet files (with Iceberg metadata) in your own bucket — giving BigQuery's DML, time travel, and access control, while the raw files stay portable to any Iceberg-compatible engine (Spark, Trino).

```
-- One-time setup: a Cloud resource connection so BigQuery can write to your bucket  
-- bq mk --connection --connection_type=CLOUD_RESOURCE --location=REGION iceberg_conn  
-- Then grant the connection's service account Storage Object Admin on the bucket.  
  
CREATE OR REPLACE TABLE `dataset.orders_iceberg` (  
  order_id INT64, customer_id INT64, region STRING, order_date DATE, status STRING  
)  
WITH CONNECTION `project.region.iceberg_conn`  
OPTIONS (  
  file_format = 'PARQUET',  
  table_format = 'ICEBERG',  
  storage_uri = 'gs://bucket/iceberg_warehouse/orders_iceberg'  
);
```

Terminology note: as of April 2026, "BigLake" for Iceberg was renamed "Lakehouse for Apache Iceberg," and "BigLake metastore" is now "Lakehouse runtime catalog." The WITH CONNECTION syntax and IAM role names are unchanged — only the console/marketing name changed.

Check it in the console: Table's Details tab shows Table type: EXTERNAL with a BigLake badge and Storage format: ICEBERG. Cloud Storage's iceberg_warehouse/ prefix contains real Parquet + metadata JSON files, readable directly by Spark/Trino outside BigQuery entirely.

2. BigQuery Pricing Model — On-Demand vs. Reservations & Slot Commitments

The problem: ShopIndex's bill is unpredictable month to month, and the data team wants nightly ETL to never compete for capacity with an executive's ad-hoc query.

	On-Demand	Editions + Reservations
You pay for	Bytes scanned per query	A committed pool of slots (query compute units)
Cost predictability	Variable, scales with usage	Fixed baseline, optional autoscale burst
Best for	Spiky, low-moderate volume	Steady, high-volume production workloads
Commitment	None	Annual/3-year commitments for best rate
Editions	N/A	Standard, Enterprise, Enterprise Plus — RLS/column security need at least Enterprise

Inspecting Reservations (Safe, Read-Only)

```
from google.cloud import bigquery_reservation_v1
client = bigquery_reservation_v1.ReservationServiceClient()
parent = f'projects/{PROJECT_ID}/locations/{LOCATION}'

for c in client.list_capacity_commitments(parent=parent):
    print(c.name, c.slot_count, c.state.name)
for r in client.list_reservations(parent=parent):
    print(r.name, r.slot_capacity)
```

Purchasing a reservation is real, ongoing money — annual/3-year lock-in. Treat the purchase commands as reference syntax to walk through, not something to execute live in a training session: `bq mk --capacity_commitment=true --edition=ENTERPRISE --plan=ANNUAL --slots=100`, then `bq mk --reservation=true --slots=100 --edition=ENTERPRISE my-reservation`, then `bq mk --reservation_assignment=true --reservation_id=my-reservation --job_type=QUERY`.

Check it in the console: BigQuery > Administration > Workload management → Slot reservations and Commitments tabs show exactly what the `list_*` calls above return, and are where you'd click "Buy slots" for a real purchasing decision.

3. IAM & Data Access Control

The problem: before row-level or column-level security matters at all, there's a coarser question — who can see this dataset or table in the first place? IAM works at three levels: project, dataset, and table/view.

Level	Grants access to	Common roles
Project	Every dataset (unless overridden)	bigquery.admin, bigquery.dataViewer, bigquery.jobUser
Dataset	Every table/view in that dataset	bigquery.dataViewer, dataEditor, dataOwner
Table/View	Just that one resource	Same roles, scoped narrower

Common gotcha: bigquery.dataViewer lets someone read table data, but bigquery.jobUser is what actually lets them run a query job at all. Granting Data Viewer without Job User is a frequent cause of "why can't this user query anything."

```
# Inspect current access on a dataset
dataset = bq_client.get_dataset('project.dataset')
for entry in dataset.access_entries:
    print(entry.entity_type, entry.entity_id, entry.role)

# Grant read-only access to just one view (not the whole dataset)
table_ref = bq_client.get_table('project.dataset.some_view')
policy = bq_client.get_iam_policy(table_ref)
policy.bindings.append({'role': 'roles/bigquery.dataViewer', 'members': {'user:x@example.com'}})
bq_client.set_iam_policy(table_ref, policy)
```

Check it in the console: Table/view page's Sharing > Permissions panel shows the same IAM bindings the code reads/writes, in the console's role-picker UI. This is how access is usually managed day to day.

How IAM, RLS, and column masking stack: IAM controls whether a user can query a resource at all; Row-Level Security (Section 8) then filters which rows they see; column masking (Section 10) determines whether specific columns show real or masked values. All three apply together, not as alternatives.

4. BigQuery ML — Training & Evaluating Models With SQL

The problem: ShopIndex's data science team wants four different models answered without ever exporting data out of BigQuery into a separate ML environment.

4.1 Logistic Regression — Predict Churn

```
CREATE OR REPLACE MODEL `dataset.churn_model`  
OPTIONS(model_type='logistic_reg', input_label_cols=['churned']) AS  
SELECT customer_segment, region, num_orders, lifetime_value, churned  
FROM training_view;  
  
SELECT * FROM ML.EVALUATE(MODEL `dataset.churn_model`);
```

4.2 Linear Regression — Predict Lifetime Value

```
CREATE OR REPLACE MODEL `dataset.ltv_model`  
OPTIONS(model_type='linear_reg', input_label_cols=['lifetime_value']) AS ...  
  
SELECT * FROM ML.PREDICT(MODEL `dataset.ltv_model`, (SELECT ... FROM new_customers));
```

4.3 K-Means — Customer Segmentation (Unsupervised)

```
CREATE OR REPLACE MODEL `dataset.customer_segments_model`  
OPTIONS(model_type='kmeans', num_clusters=3) AS  
SELECT num_orders, total_spend, tenure_days FROM customer_features;  
  
SELECT centroid_id, feature, numerical_value FROM ML.CENTROIDS(MODEL `dataset.customer_segments_model`  
);
```

4.4 ARIMA_PLUS — Revenue Forecasting (Time Series)

```
CREATE OR REPLACE MODEL `dataset.revenue_forecast_model`  
OPTIONS(model_type='ARIMA_PLUS', time_series_timestamp_col='order_date',  
        time_series_data_col='daily_revenue', auto_arima=TRUE) AS ...  
  
SELECT * FROM ML.FORECAST(MODEL `dataset.revenue_forecast_model`, STRUCT(14 AS horizon));
```

The pattern is always the same three steps: CREATE MODEL ... OPTIONS(...) AS SELECT ... to train; ML.EVALUATE to check quality; ML.PREDICT (classification/regression/clustering) or ML.FORECAST (time series) to get results. ML.EVALUATE's output columns differ by model type — precision/recall/roc_auc for classification, r2_score/mean_absolute_error for regression.

Check it in the console: Model's Evaluation tab shows precision/recall/ROC curve (classification) or forecast plots with confidence bands (ARIMA_PLUS) automatically — no charting code needed. Training tab shows the loss curve.

5. Time Travel & Change History

The problem: someone ran a bad UPDATE, or worse, dropped a table, 20 minutes ago — can the previous state be recovered without a separate backup system?

```
-- Query a table as it looked before a change (default 7-day window, configurable per dataset)
SELECT order_id, status
FROM `dataset.orders`
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 5 MINUTE)
WHERE order_id = 1001;

-- Recover an entire accidentally-dropped table
DROP TABLE `dataset.scratch_table`;
UNDROP TABLE `dataset.scratch_table`;
```

UNDROP's limit: only works within the time travel window, and only if nothing else has since been created with the exact same name — a safety net, not an infinite undo button.

Check it in the console: No dedicated console UI for time travel itself — run the FOR SYSTEM_TIME AS OF query directly in the SQL workspace and see the pre-change value returned.

6. Vector Search for RAG

The problem: a support chatbot needs to find semantically similar products given a natural-language query — not keyword matching, but "which products mean something similar."

```
CREATE OR REPLACE MODEL `dataset.embedding_model`
REMOTE WITH CONNECTION `project.region.embedding_conn`
OPTIONS (endpoint = 'text-embedding-004');

CREATE OR REPLACE TABLE `dataset.product_embeddings` AS
SELECT product_id, product_name, ml_generate_embedding_result AS embedding
FROM ML.GENERATE_EMBEDDING(MODEL `dataset.embedding_model`,
    (SELECT product_id, product_name, description AS content FROM products));

CREATE VECTOR INDEX product_index ON `dataset.product_embeddings`(embedding)
OPTIONS (index_type = 'IVF', distance_type = 'COSINE');

SELECT base.product_name, distance
FROM VECTOR_SEARCH(TABLE `dataset.product_embeddings`, 'embedding',
    (SELECT embedding FROM `dataset.product_embeddings` WHERE product_name = 'Wireless Mouse'),
    top_k => 5);
```

Needs a remote connection to a Vertex AI embedding model first (bq mk --connection --connection_type=CLOUD_RESOURCE), with the connection's service account granted Vertex AI User. At very small table sizes, the vector index may fall back to brute-force search — index behavior is tuned for much larger tables.

Check it in the console: product_embeddings table's Schema tab shows embedding as an ordinary FLOAT64 array column — not a special vector type.

7. Remote Functions & Connections

The problem: BigQuery's built-in SQL functions can't call an arbitrary external service — a currency-conversion API, a company-internal validation rule. Remote functions bridge SQL to an external HTTP endpoint (Cloud Functions/Cloud Run), reusing the same Connection resource type as Sections 1.2 and 6.

```
-- 1. Create a Cloud resource connection
-- bq mk --connection --connection_type=CLOUD_RESOURCE --location=REGION remote_conn
-- 2. Grant its service account Cloud Functions Invoker on your endpoint

CREATE OR REPLACE FUNCTION `dataset.convert_to_usd`(amount FLOAT64, currency STRING)
RETURNS FLOAT64
REMOTE WITH CONNECTION `project.region.remote_conn`
OPTIONS (endpoint = 'https://your-cloud-function-url');

SELECT order_id, convert_to_usd(total_amount, 'INR') AS amount_usd FROM orders;
```

Check it in the console: BigQuery > SQL workspace > External connections lists every connection created for BigLake, vector search, or remote functions in one place, with its service account email — the identity you grant external IAM roles to.

8. Data Lineage With Dataplex & the BigQuery Lineage API

The problem: a stakeholder asks where a number in a report actually came from. Every CREATE TABLE AS SELECT, view, and materialized view has an implicit dependency chain — tracing that by re-reading SQL doesn't scale past a handful of tables.

```
from google.cloud import datacatalog_lineage_v1
client = datacatalog_lineage_v1.LineageClient()

target = f'//bigquery.googleapis.com/projects/{PROJECT}/datasets/{DATASET}/tables/my_view'
request = datacatalog_lineage_v1.SearchLinksRequest(
    parent=f'projects/{PROJECT}/locations/{LOCATION}',
    target=datacatalog_lineage_v1.EntityReference(fully_qualified_name=target))

for link in client.search_links(request=request):
    print(link.source.fully_qualified_name, '->', link.target.fully_qualified_name)
```

Capture is automatic once the Data Lineage API is enabled — no code needed to generate it. The only manual step is querying it back. Lineage can take a few minutes up to 24 hours to actually appear — not instantaneous, so don't expect a just-run query's lineage immediately.

Check it in the console: The table/view's Lineage tab (next to Schema/Details/Preview) shows a visual dependency graph — the clearest way to see this, far more legible than the raw API text output. BigQuery, Dataflow, Dataproc, and Vertex AI all report into the same graph.

9. Row-Level Security

The problem: the APAC regional team should only see APAC orders, even though they need query access to the same orders table everyone else uses. Duplicating the table per region doesn't scale and drifts out of sync.

```
CREATE ROW ACCESS POLICY apac_only
ON `dataset.orders`
GRANT TO ('user:apac-team@example.com')
FILTER USING (region = 'APAC');
```

Check it in the console: Table's Details tab → Row access policies section lists the policy with its filter predicate and granted principals — visible even without querying as the restricted user.

10. Encryption Options — AEAD & Dynamic Data Masking

The problem: two different questions need two different mechanisms — can we encrypt a column so even someone with table access can't read it without a key (AEAD)? And can we show most users a masked version while authorized users see the real value (dynamic masking)?

10.1 Column-Level Encryption (AEAD)

```
DECLARE keyset BYTES DEFAULT KEYS.NEW_KEYSET('AEAD_AES_GCM_256');

SELECT customer_id,
       AEAD.ENCRYPT(keyset, email, CAST(customer_id AS STRING)) AS encrypted_email
FROM customers;
```

The keyset here is inline and discarded at query end. In production, store the keyset in Cloud KMS and reference it consistently so encrypted values can actually be decrypted later.

10.2 Dynamic Data Masking

Built on policy tags (the same taxonomy mechanism used for governance/classification): unauthorized users automatically see a masked value (e.g. XXXXX@XXXXX.com) with no query rewriting needed.

```
ALTER TABLE customers ALTER COLUMN email
SET OPTIONS (policy_tags = ['projects/P/locations/L/taxonomies/T/policyTags/PT']);
-- Masking rule (EMAIL_MASK, DEFAULT_MASKING_VALUE, etc.) is then set on the policy tag
-- via the console or Data Policy API.
```

Check it in the console: Governance > Knowledge Catalog > Policy tags → a tag's Data masking tab lets you pick a masking rule directly — genuinely easier to configure visually than via API.

When to use which: AEAD for true reversible encryption needed for compliance (e.g. GDPR right-to-be-forgotten via key deletion); dynamic masking when most users should simply never see raw sensitive values by default, with a small trusted group granted the unmasking role.

11. Cleanup

Every section above created real resources — tables, models, connections, a bucket. Delete the dataset (which cascades to every table/view/materialized view/model inside it), the bucket, and every connection created for BigLake/vector search/remote functions.

```
bq_client.delete_dataset(f'{PROJECT_ID}.{DATASET_ID}', delete_contents=True, not_found_ok=True)

bucket = storage_client.bucket(BUCKET_NAME)
bucket.delete(force=True)

for conn in ['iceberg_conn', 'embedding_conn', 'remote_conn']:
    # bq rm --connection --project_id=PROJECT --location=LOCATION -f CONN_NAME
    pass
```

Capacity commitments (if you actually purchased one in Section 2) cannot be deleted early. Remove assignment, then reservation, then commitment, in that order — the commitment itself only releases once its annual/3-year term ends.

Check it in the console: BigQuery > SQL workspace > Explorer, Cloud Storage > Buckets, and BigQuery > External connections should all show nothing remaining from this handbook's exercises. BigQuery > Administration > Workload management should reflect no reservations/commitments unless one was deliberately created and is being intentionally kept.

This handbook's companion notebook: `bigquery_advanced_lab.ipynb` builds and runs every section above end-to-end against your own project, using one running e-commerce dataset (customers, products, orders) throughout, with the same problem-to-solution structure and full guarded cleanup.